EECS 498 · AASE · FALL 2026

The Big Three

LECTURE 03 · SEP 8, 2026

`aider-practice` status

- Model serving?
- Aider reaching it?
- Setup friction: weights that won’t pull, a server Aider can’t reach, a machine too small?

Office hours all week, and any of us can do setup with you. **Fri Sep 11** is the gate plus all sixteen lessons.

Big Three

CONTEXT · MODEL · PROMPT

THREE LEVERS

Every AI coding session

Context

What does the model see right now?

Model

Which LLM? Capability + knowledge.

Prompt

How clearly is intent expressed?

Two out of three is **not** enough.

WHEN AI IS WRONG, ASK

Diagnostic

1. **Context** — was the file in the window?
2. **Model** — was it strong enough? recent enough?
3. **Prompt** — was intent clear?

Most students blame the model. It's usually context.

MAPPING BACK TO L02

Each lever has a mechanic

- **Context** → context window economics. Bigger isn’t free.
- **Model** → capability vs knowledge axes.
- **Prompt** → narrowing the sampling distribution.

The Big Three isn't arbitrary. It's the framework on top of the mechanics.

Context

THE LEVER YOU CONTROL MOST DIRECTLY

FOUR SOURCES

What’s in your context

- **Explicitly added files** — what you `/add`
- **Repo map** — auto-generated structure summary
- **Chat history** — accumulates until `/clear`
- **System prompt** — Aider’s instructions to the model

Adding everything is **worse** than adding the right things.

THE SKILL

Surgical context

```
For any task, ask:
  - Which files will this CHANGE?
  - Which files does the model need to READ?
  - Which files are read-only REFERENCE?

Add only those. /drop the rest. /tokens often.
```

Context is design, not setup.

READING THE BUDGET

Where your tokens go

```
/tokens

  repo map              1,800
  chat history          3,100
  main.py               6,000
  arg_parse.py          1,300
  ---------------------------
  total                12,200   of 32,768
```

Check it before and after anything big. It is the only honest view of the window.

TWO PITFALLS

Where students get stuck

Missing context

arg_parse.py not /add-ed → model only edits main.py → broken result.

Chat history bleed

Task B inherits assumptions from Task A. Fix: /clear.

CHAT BREAK · 5 MIN

Ten files, one function to change, three callers. Which files do you add?

Model

MATCH CAPABILITY TO TASK

FOUR DIMENSIONS

Choosing

- **Reasoning ability** — capability from L02
- **Speed** — usually correlated with capability
- **Cost** — dollars on a vendor, RAM and seconds on your own box
- **Context window size** — but bigger ≠ better (L02!)

THE ENGINEERING QUESTION

Right model for this job

One-line bug fix

A cheap, fast model clears the bar. Capability spent here buys nothing.

Three new files, interdependent

The strongest model you have. Getting it wrong costs more than the call.

Benchmarks (`aider.chat/docs/leaderboards/`) proxy for your task. They don't replace it.

APPLY RULE OF THUMB

Start on the model that fails

```
Start on 4b. Move to 9b when it stops teaching you
and starts merely costing you an afternoon.

A model that covers for your mistakes
hides the mistakes you need to see.
```

Switch with `/model ollama_chat/qwen3.5:9b`. Files and history persist; the new model reads all of it fresh.

Prompt

SPECIFY INTENT PRECISELY

TWO COMPONENTS

Instruction and intent

- **Instruction** — what you want done
- **Intent** — the reason you want it done that way

Leave the intent out and the model fills it from training-data priors.

Vague

```
Add another way
to output the data.
```

Model guesses. Maybe text. Maybe print(). May skip CLI flag. Unpredictable.

Specific

```
CREATE output_format.py:
  format_as_html(...) -> str
UPDATE main.py: ADD
  html to --output-format
```

Predictable. Repeatable. First-try success.

What > How.

Live Demo

CONTEXT AS ENGINEERING DECISION

Demo flow

1. Aider with only `main.py`. Prompt: `--output-format` flag.
2. It invents an `arg_parse.py` it has never seen. Decline it.
3. Run the program. Clean diff, `AttributeError`.
4. `/add arg_parse.py`. Same prompt. Coherent, and it runs.
5. `/tokens`: 1,746 → 1,998 → 3,962 for the whole `src/`.
6. `/model ollama_chat/qwen3.5:4b`. Same prompt. Compare.

The codebase is yours: `materials/demos/transcript-analytics`. Run the demo again at home.

TAKE HOME

Three questions for you

1. 50-file project, change auth across API layer — how do you /add?
2. Why does adding **all** files make results *worse*?
3. Model switch mid-session — what changes?

Most "the AI is wrong" failures collapse to question 1 or 2.

THE BIG THREE

What to take away

- **Context, model, prompt.** Three levers, every session you will ever run
- **Context is usually the problem.** Most "the AI is wrong" is missing context
- **Model choice is technical**, not a preference. Match capability to task
- **Prompts say what, not how.** Named file, named function, stated outcome

Two out of three is not enough. Ask the question before you retype the prompt.

WHAT'S NEXT

Lab01 + the build

- **Fri Sep 11** — setup gate, and all sixteen lessons due
- **Mon Sep 14 / Tue Sep 15** — Lab01: the pivot lab
- **The build** — seven features, weeks 3 and 4. Starts after Lab01, spec graded with it
- **L04** Thursday — Spec-Driven Coding

Use today's diagnostic in Lab01 when things break. *Context, model, or prompt?*

Questions?